Языки описания аппаратуры (HDL). Причины появления. Преимущества

HDL (Hardware Description Languages) — это языки, на которых описывают цифровую аппаратуру (логические схемы, блоки, процессоры) так, чтобы её можно было **смоделировать**, **проверить** и часто **автоматически синтезировать** в реальные схемы (ASIC/FPGA). Примеры: **VHDL**, **Verilog**, **SystemVerilog** (де-факто основные в индустрии).

Почему HDL вообще появились (причины)

1. **Рост сложности схем**
   - Ручные схемы/чертежи и “склейка” логики на уровне вентилей перестали масштабироваться: слишком много элементов, связей и вариантов.
2. **Нужна проверка до изготовления**
   - Ошибка в микросхеме/плате дорого стоит (время, деньги, партии). HDL позволили делать **симуляцию** и находить ошибки **до** производства.
3. **Автоматизация проектирования (EDA)**
   - Появились инструменты, которым нужен формальный вход: симуляторы, синтезаторы, статический тайминг-анализ, формальная верификация.
3. **Параллельность “по природе”**
   - В отличие от обычных языков программирования, HDL прямо выражают **одновременную работу** аппаратных блоков, что ближе к реальному железу.
4. **Иерархия и модульность**
   - Блоки, интерфейсы, параметризация (generics/parameters) — проще строить большие системы и переиспользовать компоненты.
5. **Формальная проверяемость**
   - Свойства/ассерты, формальная верификация, покрытие — можно математически/логически доказывать корректность отдельных аспектов.

4. **Переход от “рисования” к инженерному процессу**
   - Требовались повторяемость, контроль версий, ревью, трассируемость требований — как в софте.
5. **Потребность в переносимости и повторном использовании**
   - Блоки (IP-ядра) стало выгодно переиспользовать между проектами и технологиями.

Ключевые преимущества HDL

1. **Моделирование и отладка**
   - Можно прогонять тесты, смотреть волны сигналов, ловить гонки/неинициализированные состояния, проверять протоколы.
2. **Синтез в “железо”**
   - RTL-описание превращается в сеть вентилей и далее — в FPGA-битстрим или ASIC-нетлист. Это резко ускоряет путь от идеи к прототипу.

Особенности HDL. История. Стандарты VHDL. Процесс проектирования с VHDL

1) Особенности HDL (в отличие от обычных ЯП)

- **Параллельная семантика по умолчанию**: множество процессов/операторов работают одновременно (модель аппаратуры), а не “строчка за строчкой”.
- **Время как часть модели**: события, задержки, такт, “дельта-циклы” симулятора (важно для гонок/согласования сигналов).
- **Сигналы vs переменные**:
  - `signal` обновляется “событием” (обычно в конце дельта-цикла/по времени),
  - `variable` — мгновенно внутри процесса. Это критично для корректного RTL.
- **Многозначная логика** (в VHDL через `std_logic`): кроме `0/1` есть `U/X/Z` и т.п. для моделирования неопределённостей, трёхсостояний, конфликтов драйверов. 0x04 +1
- **Иерархия и структурность**: `entity/architecture`, компоненты, пакеты, библиотеки, конфигурации — удобно строить большие системы.
- **Строгая типизация (сильная сторона VHDL)**: меньше “магии” с векторами/арифметикой; зато нужно явно конвертировать типы (что повышает надёжность).

2) История (кратко)

- VHDL появился в рамках американской программы **VHSIC** (DoD, 1980–1990) как способ **документировать** и затем **симулировать** сложные ASIC/цифровые системы. Википедия +1
- Первая стандартизация: **IEEE 1076-1987** (VHDL-87), затем крупное обновление **1993**, дальше — ревизии 2000/2002, большой шаг **1076-2008**, и актуальная ветка **1076-2019**. Википедия +2
- Важный практический момент: стандартный `std_logic` для “реалистичной” цифровой логики вынесен в отдельный стандарт **IEEE 1164** (позже тесно интегрирован в экосистему VHDL).

3) Стандарты VHDL (что важно знать)

Базовый стандарт

- **IEEE Std 1076** — “язык VHDL” (LRM). Он позиционируется как формальная нотация для всех фаз создания электронных систем: разработка, верификация, синтез, тестирование. 0x04

Основные редакции IEEE 1076 (по годам)

- **1076-1987**, **1076-1993**, **1076-2000**, **1076-2002**, **1076c-2007** (VHPI), **1076-2008**, **1076-2019**. Википедия +1

4) Типовой процесс проектирования с VHDL (RTL-поток)

1. **Требования и микроархитектура**

   Определяете интерфейсы, частоты, латентности, разрядности, протоколы, reset/clocking.

2. **RTL-код на VHDL**
   - разделяете **комбинаторику** и **последовательную** логику,
   - параметризуете через `generic`,
   - фиксируете типы (часто `unsigned/signed` из `numeric_std`). Harvard Extensio... +1

3. **Тестбенч и верификация**
   - тестбенч (обычно несинтезируемый) генерирует такты/стимулы,
   - проверка через assert/scoreboard, покрытие, регрессии.

4. **Функциональная симуляция (RTL sim)**

   Ловите логические ошибки до синтеза (самый дешёвый этап исправлений).

5. **Синтез**

   RTL → нетлист, оценка ресурсов/частоты. Здесь всплывают ограничения синтезируемого подмножества.

6. **Constraints + Implementation**
   - тайминговые ограничения (clocks, IO delays),
   - place&route (FPGA) / physical design (ASIC),
   - **STA** и **timing closure**.

7. **Пост-синтез/пост-layout симуляции (по необходимости)**

   Проверка, что всё ещё работает с учётом задержек/инициализаций.

8. **Аппаратная отладка**

   FPGA-прототип, лог-анализ (ILA/SignalTap), уточнение требований/фиксы → цикл повторяется.

Проектный модуль VHDL. Design unit. Entity

Проектный модуль VHDL (Design unit)

Design unit — это *единица компиляции* в VHDL. Компилируется в **библиотеку** (`library work;`) и потом может использоваться другими модулями.

Виды design units

- **Primary units (первичные):**
  - `entity` (интерфейс блока)
  - `package` (общие типы/функции/константы)
  - `configuration` (привязка архитектур/компонентов)
  - `context` (набор `library/use` одним блоком)
- **Secondary units (вторичные):**
  - `architecture` (реализация entity)
  - `package body` (тела подпрограмм пакета)

Entity (суть)

Entity описывает *внешний интерфейс* аппаратного модуля:

- **generics** — параметры (разрядность, глубина, режимы и т.п.)
- **ports** — входы/выходы (тип, направление, иногда значения по умолчанию)

Entity **не описывает внутреннюю логику** — это делает `architecture` .

Entity. Блок портов. Порты in/out/inout

Entity (суть)

Entity описывает *внешний интерфейс* аппаратного модуля:

- **generics** — параметры (разрядность, глубина, режимы и т.п.)
- **ports** — входы/выходы (тип, направление, иногда значения по умолчанию)

Entity **не описывает внутреннюю логику** — это делает `architecture`.

Блок портов (port clause)

Это список сигналов интерфейса:

```vhdl
entity m is
  port(
    clk  : in  std_logic;
    a,b  : in  std_logic_vector(7 downto 0);
    y    : out std_logic_vector(7 downto 0);
    io   : inout std_logic
  );
end entity;
```

Формат: `имя : направление тип [:= значение_по_умолчанию];`

Направления портов

`in`

- **Только чтение** внутри модуля (вход).
- Нельзя присваивать этому порту значение в архитектуре.
- Пример: `clk`, `reset`, входные данные.

`out`

- **Только вывод** из модуля (выход).
- Внутри модуля вы **присваиваете** `out`.
- В классическом VHDL читать `out` было нельзя/нежелательно → обычно делают внутренний сигнал и дублируют на `out` (в VHDL-2008 чтение `out` разрешено, но стиль с внутренним сигналом всё равно часто используют).

`inout`

- **Двунаправленный** порт (и читается, и драйвится).
- Нужен для шин/линий, где модуль то “говорит”, то “слушает”.
- Типичный кейс: внешняя общая линия, где надо уметь отпускать её в **Z** (tri-state), напр. I²C (SDA), параллельные шины, GPIO:

Проектный модуль architecture

`architecture` — это **реализация** сущности (`entity`): как модуль работает внутри. Это **secondary design unit**: всегда привязана к конкретной `entity`.

Шаблон

vhdl Копировать код

```
architecture rtl of my_ent is
  -- объявления (внутренние сигналы, константы, типы, компоненты)
begin
  -- описание логики (конкурентные операторы и процессы)
end architecture;
```

Что внутри `architecture`

1. **Declarative part** (между `is` и `begin`)

- `signal` (внутренние сигналы)
- `constant`
- `type`, `subtype`
- `component` (если используете component-instantiation)
- `function/procedure` (локальные)
- `alias`, атрибуты и т.п.

2. **Statements part** (после `begin`)

- **конкурентные присваивания**: `y <= a and b;`
- **процессы** `process(...)` (комбин. или тактируемые)
- **генерация** `generate`
- **инстанцирование** других модулей (`entity work.xxx ...`)
- (реже) `block`, `assert`, и др.

- У одного `entity` может быть **несколько архитектур** (например `rtl`, `behav`, `gate`), выбирают нужную:
  - по имени при инстанцировании (`entity work.e(rtl)`),
  - или через `configuration`.

Типовые стили архитектуры

- **behavioral** — “как должно быть”, удобно для модели/тестов.
- **rtl** — регистры + комбинаторика (то, что синтезируется).
- **structural** — схема из подмодулей (инстансы и связи).

Проектный модуль configuration

`configuration` — это **primary design unit**, который задаёт **привязку (binding)**:

- какие **entity/architecture** использовать,
- для каких **component instances** внутри дизайна,
- из каких **libraries** брать реализации.

Идея: *интерфейс/структура может быть одной*, а **реализации** подставляются конфигурацией (например, “быстрая” vs “точная”, “RTL” vs “gate-level”).

Зачем нужен

- Явно выбрать **архитектуру** для `entity`, если их несколько.
- Явно связать **component** → конкретный `entity(architecture)` (без “правил по умолчанию”).
- Удобно для **симуляции** (особенно когда есть разные модели одного блока).
  На практике в современных RTL-проектах часто вместо `configuration` используют **direct entity instantiation** (`u1: entity work.e(rtl) ...`) — оно проще и однозначнее.

Шаблон

vhdl

```
configuration CFG_NAME of TOP_ENTITY is
  for ARCH_NAME
    -- правила привязки для экземпляров/блоков
  end for;
end configuration;
```

Типы данных VHDL. Классы типов. Объявление типов

Типы данных VHDL: суть

VHDL **строго типизирован**: типы задают, какие значения допустимы и какие операции разрешены.

Классы типов (основные группы)

1. **Скалярные (Scalar)**
- **Enumeration** (перечисления): `bit` (`'0'/'1'`), `boolean`, `character`, и пользовательские
- **Integer** (целые): `integer`, с диапазонами
- **Real** (вещественные): `real`
- **Physical** (физические): `time` (и можно свои физические)
2. **Составные (Composite)**
- **Array** (массивы): `std_logic_vector`, `string`, свои массивы
- **Record** (записи/структуры): набор полей разных типов
3. **Access** (указатели/динамика)
- `access` типы (в синтезе почти не используют)
4. **File**
- файловые типы (для тестбенчей, не для синтеза)

Как объявляют типы

1) Подтип (`subtype`)

Ограничиваем диапазон/длину:

```vhdl
subtype byte is std_logic_vector(7 downto 0);
subtype small_int is integer range 0 to 255;
```

2) Перечисление (`type ... is (...)`)

```vhdl
type state_t is (IDLE, RUN, DONE);
signal st : state_t;
```

3) Целое с диапазоном

```vhdl
signal cnt : integer range 0 to 1023 := 0;
```

4) Массив (array)

```vhdl
type mem_t is array (0 to 255) of std_logic_vector(7 downto 0);
signal mem : mem_t;
```

5) Record (структура)

```vhdl
type pkt_t is record
  addr : std_logic_vector(15 downto 0);
  data : std_logic_vector(31 downto 0);
  we   : std_logic;
end record;

signal pkt : pkt_t;
```

6) Physical (обычно `time`)

```vhdl
constant Tclk : time := 10 ns;
```

Скалярные типы данных VHDL

Скалярные типы VHDL — это **одиночные значения** (не массивы/не записи).

1) Перечислимые (Enumeration)

- **boolean**: `true / false`
- **bit**: `'0' / '1'`
- **character**: символы
- **severity_level** (для `assert`): `note, warning, error, failure`
- **пользовательские перечисления** (например FSM):

```vhdl
type state_t is (IDLE, RUN, DONE);
signal st : state_t;
```

2) Числовые

- **integer** (целый, обычно ограничивают диапазоном):

```vhdl
signal cnt : integer range 0 to 255 := 0;
```

- **real** (вещественный; в RTL почти не синтезируют, чаще для моделирования)

3) Физические (Physical)

- **time** (время — основной физический тип):

```vhdl
constant Tclk : time := 10 ns;
```

Агрегатные (составные) типы данных VHDL

Агрегатные (составные) типы VHDL — это типы, которые **состоят из нескольких элементов**.

1) Array (массивы)

Элементы **одного типа**, индексируются диапазоном.

- Встроенные/стандартные примеры: `std_logic_vector`, `string`
- Пользовательский массив:

vhdl Копироват

2) Record (запись/структура)

Поля могут быть **разных типов** — удобно объединять сигналы интерфейса/пакета.

```vhdl
type pkt_t is record
  addr : std_logic_vector(15 downto 0);
  data : std_logic_vector(31 downto 0);
  we   : std_logic;
end record;

signal pkt : pkt_t;
```

3) Важное про “агрегаты” (aggregate literals)

Для массивов/record можно задавать значения **агрегатом**:

```vhdl
v   <= (others => '0');                 -- массив
pkt <= (addr => x"0000", data => (others=>'0'), we => '0');  -- record
```

Файловые типы данных

Файловые типы в VHDL — это **специальный класс типов**, предназначенный для **ввода/вывода при моделировании** (тестбенчи, обмен данными с файлами). В синтезе почти всегда **не поддерживаются**.

Что такое file type

Файловый тип задаёт: файл — это **последовательность элементов** некоторого типа.

Самый популярный файловый тип: `text`

Он определён в пакете `std.textio` и используется для чтения/записи текстовых строк через процедуры `readline/writeline` и `read/write`.

Подключение:

```vhdl
use std.textio.all;
```

Объявление файлового объекта

Файл объявляют как объект:

```vhdl
file f_in  : text open read_mode  is "in.txt";
file f_out : text open write_mode is "out.txt";
```

Режимы открытия:

- `read_mode` — чтение
- `write_mode` — запись (перезапись)
- `append_mode` — дозапись в конец

Важно

- Файловые типы и file I/O — **для симуляции/тестбенча**, не для RTL-синтеза.
- Для `std_logic_vector` в текстовых файлах часто используют пакеты вроде `std_logic_textio` (зависит от симулятора/библиотек), чтобы удобно читать/писать двоичные/hex значения.

Если скажешь, у тебя цель **тестбенч** или “как читать std_logic_vector из файла”, дам короткий шаблон именно под это.

Подтипы. Разрешение подтипа

Подтипы (subtype) — что это

Подтип в VHDL — это *тот же самый тип*, но с **ограничением (constraint)**: диапазона, размера, индексов и т.п.
Подтип **не создаёт новый тип**, а “сужает” существующий.

Зачем нужны

- читаемость (`byte` , `word` , `addr_t`)
- безопасность (ограничить диапазон счётчика, ширину шин)
- меньше ошибок с “не теми” диапазонами

Как объявляют

vhdl Копиров

```vhdl
subtype byte_t is std_logic_vector(7 downto 0);
subtype cnt_t  is integer range 0 to 255;
```

“Разрешение” подтипа (resolved subtype)

Это про **разрешение конфликтов при нескольких драйверах** на одном сигнале.

Идея

- **Unresolved** (неразрешаемый) тип/подтип: *должен иметь максимум один драйвер*.
  Если драйверов >1 — ошибка/конфликт (очень полезно ловить баги).
- **Resolved** (разрешаемый) подтип: допускает несколько драйверов, а итоговое значение вычисляет **функция разрешения (resolution function)**.

Пакет std_logic_1164

`std_logic_1164` — это **стандартный пакет IEEE 1164**, который ввёл “нормальные” логические типы для цифрового дизайна в VHDL (то, что используют почти во всех проектах).

Как подключают

vhdl Копировать код

```
library ieee;
use ieee.std_logic_1164.all;
```

Что даёт пакет (главное)

1) 9-значная логика

Типы основаны на значениях:

- `'U'` — неинициализировано
- `'X'` — неизвестно (конфликт/неопределённость)
- `'0'`, `'1'` — логический 0/1
- `'Z'` — высокое сопротивление (tri-state)
- `'W'` — слабое неизвестно
- `'L'`, `'H'` — слабый 0 / слабая 1
- `'-'` — “don’t care” (не важно)

2) Основные типы

- `std_ulogic` — **unresolved** (разрешён только **один драйвер**)
- `std_logic` — **resolved subtype** от `std_ulogic` (можно **несколько драйверов**, есть функция разрешения конфликтов)
- `std_ulogic_vector`, `std_logic_vector` — векторные версии

Практика:

- внутри модуля часто ок `std_ulogic(_vector)` (ловит случайные два драйвера),
- на верхних портах/шинах — `std_logic(_vector)` (нужен tri-state/разрешение).

3) Логические операции и полезные функции

Пакет определяет `and/or/xor/not` и т.п. для этих типов и векторов, а также типовые функции вроде:

- `rising_edge(s)` / `falling_edge(s)` для тактирования,
- проверки/преобразования (в зависимости от версии/реализации пакета в инструментах).

Объекты данных. Константы

В VHDL “объект данных” — это то, что **хранит значение**. Основные классы:

1. **Constant** — константа (не меняется)
2. **Signal** — сигнал (соединение/провод, основа аппаратной модели)
3. **Variable** — переменная (локальная внутри процесса/подпрограммы)
4. **File** — файловый объект (для тестбенча)

Объявляются в декларативной части (`entity/architecture/package/process` — где разрешено).

Синтаксис

```vhdl
constant NAME : type := value;
```

Примеры

1) Числовая константа

```vhdl
constant N : integer := 8;
```

Переменные и сигналы

Signal — это “провод/соединение” в модели железа. Основной объект для RTL.

- Присваивание: `<=`
- Обновление **не мгновенное**: значение становится видимым после дельта-цикла (или заданной задержки).
- Может иметь несколько драйверов (для `std_logic`), а `std_ulogic` — обычно один.

Пример:

vhdl Копировать код

```vhdl
signal a, b, y : std_logic;

y <= a and b;          -- конкурентно (параллельно)
```

Переменные (variable)

Variable — “локальная” величина внутри `process`, `function`, `procedure`.

- Присваивание: `:=`
- Обновление **мгновенное** (внутри текущего выполнения процесса).
- Обычно не “видна” снаружи процесса (кроме shared variable).

Типовые применения

- **signal**: регистры, шины, входы/выходы, связи между блоками.
- **variable**: промежуточные вычисления в комбинаторном процессе, циклы, локальные счетчики в алгоритме, удобство записи.

Атрибуты VHDL. Категории. Пользовательские атрибуты

Attribute — механизм "запроса/прикрепления метаданных" к объектам/типам/сущностям: можно получить *свойство* (встроенный атрибут) или назначить *пользовательское* значение.

Категории атрибутов (по назначению)

1) Предопределённые (built-in, language-defined)

Делятся по тому, к чему применяются:

A) Атрибуты типов/скалярных

- `T'left`, `T'right`, `T'low`, `T'high` (границы)
- `T'range`, `T'reverse_range`
- `T'pos(x)`, `T'val(i)`, `T'succ(x)`, `T'pred(x)` (для enumeration/дискретных)

B) Атрибуты массивов/векторов (очень частые)

- `a'length`, `a'left`, `a'right`, `a'range`, `a'reverse_range`
- `a'ascending` (направление индекса)

C) Атрибуты сигналов (события)

- `s'event` — было ли событие (изменение) на сигнале
- `s'last_value`, `s'last_event`, `s'last_active`
- `s'stable(t)`, `s'quiet(t)` (чаще в симуляции)

Классика (старый стиль тактирования):

```vhdl
if clk'event and clk='1' then ...
```

(обычно пишут `rising_edge(clk)`)

D) Атрибуты объектов/имён

- `x'image`, `x'value` (строковое представление/парсинг для некоторых типов)

Пользовательские атрибуты (user-defined)

Их делают так:

1) Объявить имя и тип атрибута

```vhdl
attribute KEEP : boolean;
```

2) Привязать значение к конкретному объекту/имени

```vhdl
attribute KEEP of my_sig : signal is true;
```

Важно: **семантика пользовательского атрибута не задана языком**, её интерпретирует инструмент (симулятор/синтезатор). То есть сам VHDL “не знает”, что значит `KEEP` — это договорённость.

Предопределённые атрибуты типов

Предопределённые **атрибуты типов** (type attributes) в VHDL применяются как `T'attr`, где **T** — имя типа/подтипа.

1) Границы и диапазоны (дискретные и массивные типы)

- `T'left`, `T'right` — левый/правый предел диапазона
- `T'low`, `T'high` — минимальное/максимальное значение диапазона
- `T'range` — сам диапазон (например `7 downto 0` или `0 to 7`)
- `T'reverse_range` — диапазон в обратном направлении
- `T'length` — длина диапазона (кол-во значений)

vhdl Копировать код

```
subtype idx_t is integer range 3 to 7;
-- idx_t'low  = 3
-- idx_t'high = 7
-- idx_t'length = 5
```

2) Для перечислений/дискретных (enumeration, integer) — переходы и позиции

- `T'pos(x)` — порядковый номер значения `x` (0,1,2…)
- `T'val(i)` — значение типа по номеру `i`
- `T'succ(x)` / `T'pred(x)` — следующий / предыдущий элемент
- `T'leftof(x)` / `T'rightof(x)` — сосед слева/справа по порядку следования
- `T'base` — базовый тип (для подтипов)

3) Преобразование в строку (полезно в тестбенче)

- `T'image(x)` — строка с именем/значением
- `T'value("...")` — значение типа из строки

Предопределённые атрибуты агрегатов (обычно массивов)

Предопределённые атрибуты **агрегатов/массивов** в VHDL применяются к *имени массива* (сигнал/переменная/константа массива) и пишутся как `A'attr`.

1) Границы индексов и диапазон

- `A'left` / `A'right` — левый/правый предел индекса (для заданного измерения)
- `A'low` / `A'high` — минимальный/максимальный индекс
- `A'range` — диапазон индекса (например `7 downto 0`)
- `A'reverse_range` — диапазон в обратном направлении
- `A'ascending` — `true`, если индекс идёт `to`, и `false`, если `downto`

Пример:

vhdl Копировать код

```vhdl
signal v : std_logic_vector(7 downto 0);
-- v'left = 7, v'right = 0
-- v'range = 7 downto 0
-- v'ascending = false
```

2) Длина

- `A'length` — количество элементов (для измерения)

vhdl Копирс

```vhdl
constant W : integer := v'length;  -- 8
```

3) Многомерные массивы (важно!)

Для массивов с несколькими измерениями атрибуты можно писать с номером измерения:

- `A'left(1)`, `A'length(2)`, `A'range(2)` и т.д.

Предопределённые атрибуты сигналов

Предопределённые **атрибуты сигналов** в VHDL (пишутся как `s'attr`, где `s` — сигнал) в основном нужны для **моделирования событий и времени**.

1) События и последнее изменение

- `s'event` — было ли **событие** (изменение значения) на `s` в текущем шаге симуляции.
- `s'last_event` — сколько времени прошло с **последнего события** на `s`.
- `s'last_value` — **предыдущее** значение `s` перед последним событием.

Пример (классическое тактирование, “старый стиль”):

vhdl Копировать код

```
if clk'event and clk = '1' then
  q <= d;
end if;
```

2) Активность (active)

“Активность” включает не только смену значения, но и, например, повторные транзакции (зависит от модели сигнала).

- `s'active` — была ли активность на сигнале в текущем шаге.
- `s'last_active` — время с последней активности.

3) Стабильность и тишина на интервале

Эти атрибуты принимают параметр времени `t : time`:

- `s'stable(t)` — **не менялся** ли сигнал в течение последних `t`.
- `s'quiet(t)` — **не было активности** в течение последних `t`.

4) Задержанные/сэмплированные значения (часто в тестбенче)

- `s'delayed(t)` — значение `s`, **задержанное** на `t`.
- `s'stable` / `s'quiet` выше тоже часто используют в проверках.

Операторы VHDL. Объявление (декларация) операторов

В VHDL “операторы” обычно понимают как **символы/ключевые слова для выражений**:

- **Арифметические**: `+ - * / ** mod rem abs`
- **Сравнения**: `= /= < <= > >=`
- **Логические (boolean)**: `and or nand nor xor xnor not`
- **Побитовые/векторные** (на `std_logic(_vector)` через пакеты): `and/or/xor/...`
- **Сдвиги/вращения**:
  - `sll`, `srl` (логические)
  - `sla`, `sra` (арифметические)
  - `rol`, `ror` (rotate)
- **Конкатенация**: `&`
- **Принадлежность диапазону**: `in` (например `if i in 0 to 7 then`)
- (ещё есть оператор сопоставления `?=` в VHDL-2008 для “don’t care”, но на экзаменах чаще достаточно базового набора)

Объявление (декларация) операторов = перегрузка

В VHDL можно **создавать/перегружать операторы** через объявление функции с “именем-оператором”.

1) Декларация оператора (в package или declarative part)

Синтаксис: имя функции берётся в кавычки.

vhdl Копиро

```vhdl
function "+" (L, R : my_t) return my_t;
function "and" (L, R : my_t) return my_t;
function "sll" (L : my_vec_t; N : natural) return my_vec_t;
```

2) Тело оператора (в package body)

vhdl

```vhdl
function "+" (L, R : my_t) return my_t is
begin
  ...
end function;
```

Параллельные операторы. Concurrent signal assignment

В архитектуре **после** `begin` всё, что стоит напрямую (не внутри `process`), выполняется **параллельно**. Главный вид — **concurrent signal assignment**: присваивание сигналу, которое постоянно “поддерживает” связь входов и выхода.

Concurrent signal assignment (параллельное присваивание сигнала)

1) Простое (simple concurrent assignment)

vhdl Копиров

```vhdl
y <= a and b;
s <= x + 1;
```

2) Условное (conditional) — аналог if/elsif/else

vhdl

```vhdl
y <= a when sel='0' else b;
```

3) Выборочное (selected) — аналог case

vhdl

```vhdl
with sel select
  y <= d0 when "00",
       d1 when "01",
       d2 when "10",
       d3 when others;
```

Тоже обычно **мультиплексор**, но форма “case-подобная”.

---

4) (Часто рядом) Инстанцирование блоков — тоже concurrent

vhdl

```vhdl
u1: entity work.adder(rtl)
  port map (a=>a, b=>b, s=>s);
```

Concurrent: создание экземпляра компонента (component instantiation)

Это **параллельный оператор** в `architecture` (после `begin`), который вставляет внутрь текущего модуля **экземпляр другого модуля** и связывает порты сигналами.

1) Классический стиль: `component` + `port map`

Сначала объявляют компонент (обычно в declarative part архитектуры или в package):

```
architecture structural of top is
  component adder
    port(
      a, b : in  std_logic_vector(7 downto 0);
      s    : out std_logic_vector(7 downto 0)
    );
  end component;

  signal a,b,s : std_logic_vector(7 downto 0);
begin
  U1 : adder
    port map (
      a => a,
      b => b,
      s => s
    );
end architecture;
```

- `U1` — **label** (имя экземпляра)
- `adder` — имя **component**
- `port map` — сопоставление портов (лучше **named association** как выше)

3) `generic map` (параметры)

Если у блока есть generics:

```vhdl
U1 : entity work.fifo
  generic map (DEPTH => 256, WIDTH => 8)
  port map    (clk=>clk, rst=>rst, din=>din, dout=>dout);
```

Concurrent: оператор process

`process` — **параллельный оператор** архитектуры: сам процесс запускается **конкурентно** с другими concurrent-операторами, но **внутри процесса** команды выполняются **последовательно**.

Шаблон:

vhdl Копировать код

```
process (sensitivity_list)
begin
  -- sequential statements
end process;
```

1) Комбинаторный process

Идея: выходы зависят только от текущих входов.

vhdl Копировать ко

```
process(all)  -- VHDL-2008: все сигналы, которые читаются внутри
begin
  y <= a and b;
end process;
```

Правило: в комбинаторном процессе нужно **присваивать всем выходам во всех ветках**, иначе получится latch.

2) Тактируемый (синхронный) process

Идея: регистры/триггеры на фронте такта.

vhdl

```
process(clk)
begin
  if rising_edge(clk) then
    q <= d;
  end if;
end process;
```

Concurrent: generate

`generate` — **параллельная конструкция** для *порождающего описания*: она создаёт **много одинаковых** (или условных) concurrent-операторов на этапе elaboration (до симуляции/синтеза). Это не цикл “во времени”, а **размножение аппаратуры**.

1) `for ... generate` (размножить N экземпляров)

Пример: N-битный регистр (структурно)

```vhdl
gen_ff : for i in 0 to N-1 generate
begin
  ff_i : entity work.dff
    port map (clk=>clk, d=>d(i), q=>q(i));
end generate;
```

2) `if ... generate` (условно включить/исключить железо)

```vhdl
gen_parity : if USE_PARITY generate
begin
  p <= xor_reduce(data);
end generate;
```

Если `USE_PARITY=false`, этого блока физически “нет”.

3) `case ... generate` (выбор одной из реализаций)

```vhdl
gen_impl : case MODE generate
  when 0 =>
    -- одна реализация
  when 1 =>
    -- другая
  when others =>
    -- запасная
end generate;
```

Concurrent: assert и block

Concurrent `assert`

`assert` может стоять **параллельно** в архитектуре (после `begin`) и тогда проверка “живёт сама по себе”, реагируя на изменения выражения во времени (в симуляции).

Форма:

vhdl Копировать код

```vhdl
assert condition
  report "message"
  severity warning;  -- note | warning | error | failure
```

Concurrent `block`

`block` — это **параллельная “обёртка”** для группы concurrent-операторов:

- даёт **локальную область видимости** (можно объявлять локальные сигналы/константы),
- помогает структурировать дизайн,
- позволяет задать локальные `generic/port` (редко в современном RTL),
- исторически использовался вместе с configuration/binding.

Шаблон:

vhdl Копиров

```vhdl
label : block
  -- declarative part (локальные объявления)
begin
  -- concurrent statements
end block label;
```

Sequential: if, case, пустой оператор

Sequential `if`

Используется **внутри process / function / procedure**.

Шаблон:

```vhdl
if cond then
  -- statements
elsif cond2 then
  -- statements
else
  -- statements
end if;
```

Sequential `case`

Тоже только внутри sequential-кода.

Шаблон:

```vhdl
case expr is
  when v1      => ...
  when v2|v3   => ...
  when others  => ...
end case;
```

Пустой оператор (`null`)

`null;` — **ничего не делает**. Нужен, когда синтаксически требуется statement, но действий нет.

Sequential: операторы цикла

Циклы бывают **только в sequential-коде** (внутри `process`, `function`, `procedure`). Это не “параллельность”, а повторение команд при выполнении процесса.

---

1) `loop ... end loop` (бесконечный/с выходом)

vhdl Копир

```vhdl
loop
  exit when done = '1';
  -- statements
end loop;
```

2) `while ... loop`

vhdl

```vhdl
while i < 10 loop
  i := i + 1;
end loop;
```

---

3) `for ... loop` (самый частый в RTL)

vhdl

```vhdl
for i in 0 to 7 loop
  sum := sum xor v(i);
end loop;
```

```
for i in 0 to 7 loop
  sum := sum xor v(i);
end loop;
```

- `i` — локальный индекс (не надо объявлять отдельно)
- диапазон может быть `to` или `downto`, или `'range` массива:

```vhdl
for i in v'range loop
  ...
end loop;
```

Управляющие операторы внутри циклов

- `exit;` или `exit when condition;` — выйти из цикла
- `next;` или `next when condition;` — перейти к следующей итерации

Sequential: изменение последовательности циклов. Присваивания signal/variable

Изменение последовательности выполнения в циклах (sequential)

`exit` — выход из цикла

- `exit;` — выйти сразу
- `exit when cond;` — выйти, когда условие истинно

```vhdl
for i in 0 to 15 loop
  exit when a(i)='1';
end loop;
```

`next` — перейти к следующей итерации

- `next;`
- `next when cond;`

```
for i in 0 to 15 loop
  next when mask(i)='0';  -- пропустить итерацию
  cnt := cnt + 1;
end loop;
```

Метки циклов (для вложенных циклов)

Чтобы выйти/перейти не из внутреннего, а из внешнего цикла:

```vhdl
outer: for i in 0 to 3 loop
  inner: for j in 0 to 3 loop
    exit outer when stop='1';  -- выйти из outer
    next outer when skip='1';  -- следующая итерация outer
  end loop inner;
end loop outer;
```

Присваивания в цикле:

- `variable :=` — **сразу** меняется, удобно для сумм/накопления внутри цикла.
- `signal <=` — **позже** обновляется; много `<=` одному сигналу за проход → сработает **последнее** (накапливать так нельзя).

Sequential: оператор ожидания wait

`wait` — **последовательный оператор** (только внутри `process`) для приостановки процесса до события/условия/времени. В RTL для синтеза почти не используют, это в основном **testbench**.

Основные формы `wait`

1) По времени

vhdl Копировать к

```
wait for 10 ns;
```

2) До события на сигнале

vhdl Копировать к

```
wait on clk;          -- ждать любое изменение clk
```

3) До условия

vhdl

```
wait until ready='1';
```

Часто вместе с фронтом:

vhdl

```
wait until rising_edge(clk);
```

4) Комбинации

vhdl

```
wait on a, b until en='1' for 100 ns;
```

Подпрограммы: функции

Функция — подпрограмма, которая:

- **возвращает одно значение** (`return`)
- может иметь входные параметры (`in` , по умолчанию `in`)
- обычно **не должна иметь побочных эффектов** (для синтезируемого RTL: не писать в сигналы, не делать `wait` , не использовать file I/O)

Объявление (декларация)

В `package` , `architecture` , `process` (где разрешено):

vhdl Копировать код

```vhdl
function fname (a : in integer; b : in integer) return integer;
```

Определение (тело)

vhdl

```vhdl
function fname (a : in integer; b : in integer) return integer is
begin
  return a + b;
end function;
```

Подпрограммы: процедуры

Процедура — подпрограмма, которая:

- **не возвращает значение** (нет `return <expr>`)
- может **изменять параметры** через `out` / `inout`
- удобна для “действий”: присваиваний, общих шаблонов, тестбенча

Декларация

```vhdl
procedure pname (a : in  ...; b : out ...; c : inout ...);
```

Тело

```vhdl
procedure pname (a : in ...; b : out ...; c : inout ...) is
begin
  ...
end procedure;
```

Верификация устройства. Моделирование

Верификация = доказать, что схема делает **то, что должна**, во всех важных режимах, *до* изготовления/прошивки.

Что проверяют:

- **Функциональность** (правильные выходы на правильные входы)
- **Протоколы интерфейсов** (ready/valid, AXI, SPI, I²C и т.п.)
- **Границы/углы** (reset, переполнения, пусто/полно, таймауты, редкие комбинации)
- **Синхронизацию** (такт, reset, CDC, метастабильность — отдельная тема)
- **Тайминг** (успевает ли на нужной частоте)

Инструменты/подходы:

- **Симуляция** (основная)
- **Assertions (assert)** — автоматические проверки свойств
- **Coverage** — насколько хорошо “потрогали” дизайн тестами
- **Формальная верификация** (доказательство свойств без перебора тестов)
- **Equivalence checking** (RTL ≡ netlist после синтеза)

Моделирование = прогон схемы во времени на симуляторе, чтобы увидеть сигналы и поймать ошибки.

Уровни моделей:

- **Behavioral** — “как должно быть” (быстро, удобно для эталона)
- **RTL** — регистры + комбинаторика (то, что синтезируется)
- **Gate-level (netlist)** — вентильный уровень, иногда с задержками (SDF) для проверки после синтеза/разводки

Что запускают:

- **Тестбенч**: генерирует такт/reset/стимулы, содержит эталон/проверки.
- Типовая схема: *stimulus → DUT → monitor/scoreboard → assert/лог ошибок*.

Коротко: **верификация — это процесс**, а **моделирование — главный инструмент в этом процессе**. Если хочешь — дам супер-краткий шаблон тестбенча VHDL (clock/reset + 2–3 assert).

Тестовые модули (testbench)

Testbench — это VHDL-модуль для **проверки DUT** (device under test). Он обычно **не синтезируется**.

Состав testbench (суть)

- **DUT**: инстанс проверяемого модуля.
- **Генераторы**: такт (`clk`), сброс (`rst`), входные стимулы.
- **Проверки**: `assert`, сравнение с эталоном (scoreboard), лог ошибок.
- (опц.) **файлы/модели окружения**: `textio`, модели памяти, шины.

Типовая структура

1. `entity tb is end;` — обычно без портов
2. `architecture sim of tb is`
   - сигналы для связи с DUT
3. `begin`
   - инстанс DUT
   - `process` для clock/reset
   - `process` для стимулов
   - `assert`/мониторы

Стили описания цифровых устройств

Основные **стили описания цифровых устройств** в HDL (в т.ч. VHDL):

- **Поведенческий (behavioral)**
  Описываешь *что должно получиться*, ближе к алгоритму. Удобно для моделей/эталонов, но не всё синтезируется.
- **RTL (Register-Transfer Level)**
  Самый основной для синтеза: **регистры (по такту) + комбинаторика между ними**. Обычно: тактируемые `process` + комбинаторные `process`/concurrent.
- **Структурный (structural)**
  “Схема из блоков”: инстанцирование компонентов/модулей и соединения между ними (как netlist, но на уровне модулей).
- **Вентильный/схемный (gate-level)**
  На уровне логических элементов/нетлиста (часто результат синтеза). Используют для post-synth/post-layout проверки с задержками.

Уровни абстракции: транзисторный уровень

Транзисторный уровень (transistor-level) — самый “низкий” уровень абстракции: схема описывается **как соединение MOS-транзисторов** (NMOS/PMOS), с их размерами и электрическими эффектами.

Что описывают

- сеть транзисторов и соединений (pull-up/pull-down)
- питание/земля (VDD/GND), паразитики
- размеры транзисторов (W/L), пороговые эффекты
- аналоговые параметры: токи, напряжения, задержки, шумы, утечки

Где применяется

- проектирование **стандартных ячеек**, библиотек (NAND, FF, latch и т.д.)
- критические **аналоговые/смешанные** блоки
- анализ **энергопотребления**, утечек, точных задержек
- верификация “на физике” после layout (паразитики)

Моделирование/инструменты

- обычно **SPICE**/Spectre и т.п. (электрическое моделирование)
- долго, но максимально точно

Плюсы/минусы

- ✅ максимальная точность и контроль
- ❌ очень медленно и плохо масштабируется для больших цифровых систем
  Поэтому большие цифровые блоки обычно делают на **RTL/gate-level**, а транзисторный уровень оставляют для ячеек и “узких мест”.

Уровни абстракции: вентильный уровень

Вентильный уровень (gate-level) — уровень абстракции, где устройство описывается как **схема из логических элементов** (AND/OR/NOT, MUX, триггеры) или, чаще, из **стандартных ячеек** библиотеки (NAND2, DFF, AOI и т.п.) и соединений между ними.

Что является моделью

- **Netlist**: список элементов + связи (сигнальные сети)
- Элементы берутся из библиотеки (FPGA primitives или ASIC standard cells)

Откуда берётся

- обычно это **результат синтеза** RTL → netlist
- после place&route может добавляться **тайминг** (SDF)

Для чего используют

- **Проверка эквивалентности**: RTL ≡ netlist (LEC)
- **Пост-синтез / пост-лейаут симуляция** (иногда с задержками)
- Оценка реальных задержек/гоночных эффектов, reset/init, поведения библиотечных ячеек

Уровни абстракции: уровень регистровых передач (RTL)

RTL (Register-Transfer Level) — уровень абстракции, где цифровое устройство описывают как:

- **регистры** (состояние), которые обновляются **по такту**;
- **комбинаторная логика** между регистрами, которая вычисляет “следующее состояние” и выходы.

Что описывают на RTL

- какие сигналы/данные **регистрируются** на `rising_edge(clk)` (или `falling_edge`)
- как считаются `next_state`, `next_data`
- FSM, счетчики, конвейеры, datapath + control

Типовые конструкции VHDL для RTL

- **Тактируемый процесс** (регистры):

```vhdl
process(clk)
begin
  if rising_edge(clk) then
    q <= d;
  end if;
end process;
```

- **Комбинаторика** (next-state/выходы):

```vhdl
process(all)
begin
  d <= a and b;
end process;
```

Уровни абстракции: системный уровень

Системный уровень (system-level) — самый высокий уровень абстракции: описывают не вентильную логику, а **всю систему как набор блоков и их взаимодействие** (функции, потоки данных, протоколы, производительность).

Что описывают на системном уровне

- **архитектуру**: какие подсистемы есть (CPU, DMA, память, периферия), как связаны (AXI/AHB, NoC)
- **поведение и алгоритмы**: что делает система, без деталей регистров/тактов
- **потоки данных** и сценарии использования
- **требования**: производительность (latency/throughput), пропускная способность, энергопотребление “в целом”
- **разбиение HW/SW**: что в железе, что в софте

Как обычно моделируют

- “Исполняемые модели” для быстрой проверки идей: транзакционный уровень (**TLM**), высокоуровневые модели (часто SystemC, C/C++, MATLAB/Simulink).
- В VHDL это могут быть **behavioral-модели** блоков и тестовые окружения, но для настоящего system-level чаще берут другие средства.

Зачем нужен

- быстро проверить архитектурные решения до RTL
- оценить узкие места, буферы, очереди, арбитраж
- снизить риск: меньше дорогих переделок на поздних стадиях

Связь стилей описания и уровней абстракции. Задачи проектирования

Связь “стиль описания” ↔ “уровень абстракции”

- **Транзисторный уровень** ↔ стиль *транзисторный/схемотехнический* (SPICE): описывают NMOS/PMOS, размеры, электрические эффекты.
- **Вентильный (gate-level)** ↔ стиль *структурный* (netlist): элементы/стандартные ячейки + соединения.
- **RTL** ↔ стиль *RTL* (регистр + комбинаторика): тактируемые процессы + комбинаторные связи.
- **Системный уровень** ↔ стиль *поведенческий/архитектурный*: блоки как “чёрные ящики”, транзакции, алгоритмы, производительность.

Упрощённо: чем выше уровень абстракции, тем чаще **behavioral**; чем ниже — тем больше **структурный/netlist**.

Задачи проектирования на разных уровнях

Системный

- выбрать архитектуру, интерфейсы, HW/SW-разделение
- оценить throughput/latency, буферы, арбитраж, память

RTL

- разработать микроархитектуру: FSM, конвейеры, datapath/control
- написать синтезируемый код, сделать тестбенчи, закрыть функциональные требования

Gate-level

- проверить эквивалентность RTL ↔ netlist
- пост-синтез/пост-layout проверки, влияние задержек (при необходимости)

Транзисторный

- проектировать стандартные ячейки/критические аналоговые узлы
- точная мощность/задержки/утечки, проверка после layout с паразитиками

Синтезируемые конструкции VHDL. FSM на VHDL

Синтезируемые конструкции VHDL (что “идёт в железо”)

Обычно синтезируется (RTL-подмножество):

- **Entity/architecture**, `generic`, `port`
- **Сигналы/константы**, простые типы + `std_logic(_vector)`, `signed/unsigned` (`numeric_std`)
- **Concurrent присваивания**: `y <= expr;`, `with select`, `when else`
- **Process**
  - тактируемый: `if rising_edge(clk) then ... end if;`
  - комбинаторный: `process(all)` + полные присваивания
- **Операторы**: логика, сравнения, `+ - *` (ограниченно по ресурсам), сдвиги, `&`
- **if/case**, **for loop** с *константными/ограниченными* границами (обычно разворачивается)
- **generate** (`for/if/case generate`)
- **RAM/ROM inference** (если соблюсти шаблон под инструмент)

FSM на VHDL (шаблон “как надо”)

Вариант 1: классика “2 процесса” (рекомендуется)

- **1.** регистр состояния
- **2.** комбинаторика next-state/выходов

vhdl

```
library ieee;
use ieee.std_logic_1164.all;

entity fsm_ex is
  port (
    clk   : in  std_logic;
    rst   : in  std_logic;
    start : in  std_logic;
    done  : in  std_logic;
    y     : out std_logic
  );
end entity;
```